Proyección - π

π columna1,columna2,... (tabla) - Sirve para proyectar columnas

Selección - σ

σ [condicion] (tabla) - Sirve para seleccionar registros

Renombramiento - ρ

ρ columna_orig → nombre_nuevo (tabla)
rDEPT(dep,desc)

Agrupar - γ

γ columna1,columna2,... ; operacion → nombre_nuevo (tabla)

La operacion puede ser count, sum, average, min, max.

Full outer join - □

Tabla 1 □ Tabla 2

Une las tablas por las claves foraneas. Estas se deben especificar despues del operador
Ej: Bodega □ BodegaID=BodegaDestinoID Envio

A menos que tengan el mismo nombre

Semi Joins - □ □

Funciona de igual forma que el join, pero siempre traeran incluso los valores nulos de el conjunto al que
apunten.

DDL

Create

Ejemplo de crear una tabla:

```
create table if not exists usuario(
    id INT primary key auto_increment,
    nickname varchar(45) not null unique,
    email varchar(45) not null,
    fechaRegistro datetime
);

create table [if not exists] nombre_tabla (
    nombre_columna tipo_de_dato (...),            //Tipo de dato enum(enum1,enum2,...)
    """
    """
);
```

En esos ... podría ir:
- primary key, si el atributo es clave primaria
- auto_increment, si queremos que el atributo se incremente solo
- unique, si queremos que sea un valor unico en la tabla
- not null, si no es un campo requerido
- default, para ponerle un valor por defecto
- comment, para agregarlo un comentario a la columna
- constraint nombre_constraint check (condicion), para declarar una condicion en esa columna

Para agregar una foreign key:
constraint nombre_constraint foreign key (columna_de_la_tabla) references

columna_foranera(tabla)

Para primary key compuesta:
constraint nombre_constraint primary key (columna1,columna2,...)

Alter
alter table nombre_tabla ...

- Se puede usar add si se quiere agregar una condición, una nueva columna o un nuevo constraint.

Con condición me refiero a check (condicion)

- Se puede usar modify para modificar una columna.

Drop
Elimina una tabla

drop table if exists nombre;

DML
insert, Agrega datos a una tabla

```
insert into tabla nombre_tabla (tupla de columnas)
    values (tupla de valores),(),(),...;
```

- Si se tiene un valor auto incremental y no se lo inserta el programa igual le asignara un valor y lo ira
sumando.
- Si por ejemplo insertamos varios registros y nuestro valor incremental va por 3, luego ingresamos
manualmente este valor pero le ponemos por ejemplo 10. Entonces la siguiente vez que el programa vaya a
seguir autoincrementando empezara desde el 11.

select, consultar datos de una tabla

```
select columna1,columna2,... from table nombre_tabla; //Selección normal

select columna1,columna2,... from table nombre_tabla where condicion;//Selección con condicion
```

Se puede usar el comodin * para decir que queremos todas las columnas de la tabla

Ej de selección:

```
select *
from payment
where amount not in (3.99, 5.99, 9.99) // En este caso se esta buscando los registros donde el monto
no se encuentre en esa tupla de valores, obviamente tambien se puede usar in para ver el monto que si
se encuentre.

select *
from rental
where return_date is null; //En este caso se esta buscando los registros donde la fecha de retorno
es nula

select *
from payment
-- where payment_date
between '2005-06-15 00:00:00' and '2005-06-30 23:59:59'
    and amount > 8
-- where amount between 5 and 10
-- where  amount >= 5 and amount <=10

//En el caso de arriba se tienen tres formas diferentes de validar un rango usando la función between o
```

```
usando los operadores logicos de mayor igual y menor igual.

select first_name,last_name
from customer
where first_name like 'F%' and store_id = 1

// La palabra like ayuda a hacer comparaciones de strings se puede usar el comodin % que significa
lo que sea en el ejemplo de arriba 'F%' significa que empiece en F y le siga lo que sea.

// También se puede usar el guión bajo para indicar que cantidad de caracteres queremos a continuación.
select * from actor
-- where last_name like '%A___';  //Que empiece con lo que sea, tenga una A en medio y tenga 3 caracteres despues
-- where last_name like '_A___'; //Que tenga un caracter antes de la A y 3 caracteres despues
-- where last_name like 'P____'; //Que empiece con P y tenga 4 caracteres despues de la P
-- where last_name like '%PE%'; //Que empiece y termine con lo que sea pero que tenga PE en medio
-- where last_name like 'PE%' or first_name like 'PE%'; //Que empiece con PE y termine en lo que sea
-- where last_name like '%PE'; //Que empiece con lo que sea y termine con PE



select * , year(payment_date), month(payment_date)
from payment
where month(payment_date) between 5 and 7

//La funcion year y month sacan el año y el mes de una
//fecha, estas se agregan como columna a la tabla

TIMESTAMPDIFF(formato, fecha_pequeña, fecha_mayor)
// Esta funcion saca la diferencia entre fechas.

select
CASE
    WHEN dayofweek(rental_date) =1 THEN "Domingo"
    WHEN dayofweek(rental_date) =2 THEN "Lunes"
    ELSE dayofweek(rental_date)
END
from

//Funciona como un switch siempre empieza con case y termina en end, dentro tiene anidadas varias palabras
when y al final un else.

Las funciones lower() y upper() sirven para poner en minusculas y mayusculas respectivamente.

select lower(first_name), last_name
from actor;

//No modifica la tabla original solo esta haciendo una proyección

SELECT distinct district FROM address; // distinct es para que no se repita valores

select *, amount + iva as total
from payment
order by month(payment_date) asc ,  amount desc;

//Se puede hacer operaciones y se usa la palabra as para indicar el nombre que va a tener esa columna
en la proyeccion.
// Se puede hacer un ordenamiento con order by columna asc o desc, columna2 asc o desc, ...;
// Siempre empieza ordenando desde la columna de la izquierda.
```

```
186
187
188  update, modificar datos de una tabla
189
190  Ej:
191
192  update actor
193  set first_name = lower(first_name);
194
195  update nombre_tabla set columna = cambio, columna2 = cambio;
196
197  Ademas se puede agregar condiciones
198
199  update nombre_tabla set columna = cambio, columna2 = cambio where condicion;
200
201  //La funcion concat sirve para unir strings, se puede poner la cantidad que se desee y
     se unen todos.
202  concat(first_name, ' ', last_name)
203
204
205  delete, elimina datos de una tabla
206
207  delete from nombre_tabla; //Delete normal
208
209  delete from nombre_tabla where condicion; //Delete con condicion
```